PRD.md — BootPulse

Product: BootPulse — cash-flow, runway, and scenario-planning dashboard
Version: 1.0 (Phase 1 MVP)
Document owner: Lead Product Manager / Technical Architect
Status: IDE-ready specification — validated concept
Last updated: 2026-10-16

---

1. Executive Summary

1.1 Core Value Proposition

BootPulse is a **desktop-first cash-flow, runway, and scenario-planning dashboard** for solo founders and small bootstrapped teams who have no dedicated finance function.

It answers three questions in under a minute:

1. **How long can my business run?** — runway in months and the projected zero-cash date.
2. **Can I afford this decision?** — hiring a freelancer, buying software, or changing prices modelled side-by-side against the baseline.
3. **What actually changed?** — a monthly forecast table plus a decision log so the reasoning behind each financial move is recorded and reviewable.

The differentiator is **transparency over automation**. Every number is derived from a visible, editable formula. The product does not ask founders to trust a black box — it shows the arithmetic.

1.2 Target User Personas

Persona	Profile	Primary job-to-be-done	Current workaround	Success signal
Priya — Solo Founder (Primary)	Non-technical founder of a 1-person software studio. Revenue is lumpy.	"Tell me how many months I have left before I need to cut costs."	Google Sheets with hardcoded formulas that break silently.	Opens BootPulse weekly; runway number matches her expectation within ±5%.
Marco — Bootstrapped Team Lead (Primary)	Runs a 3–5 person team. Two part-time contractors.	"Can I add a contractor for 3 months without going below 2 months of runway?"	Ad-hoc calculator + gut feel.	Runs at least 2 scenarios before each hiring decision.
Dana — Fractional Adviser (Secondary, Phase 2)	Advises 4–6 early-stage founders.	"Give me a read-only view of each client's runway."	Screenshots and CSV exports.	Requests adviser-tier access.

Explicitly out of scope as a user: finance teams, accountants, CFOs, funded scale-ups needing GAAP reporting.

1.3 Monetisation Mechanics

Monthly subscription, tiered, billed via **Stripe Checkout in test mode only** for this assignment. Feature gating is enforced server-side by reading the subscription tier from the `businesses/users` record — never from client state.

Tier	Price (test mode)	Businesses	Forecast horizon	Scenarios	Adviser access	Support
Starter	$12 / month	1	6 months	2	—	Self-serve docs
Growth	$29 / month	3	18 months	10	—	Email
Scale	$59 / month	10	36 months	Unlimited	Read-only adviser link	Priority email

Gating rules

- Creating a business, scenario, or extending a forecast beyond the tier horizon requires an **active subscription of the required tier**.
- Downgrade is allowed at any time; data above the new limit becomes **read-only, never deleted**.
- A 14-day trial maps to the **Growth** tier limits; expiry downgrades to Starter read-only until checkout completes.

  All prices, product IDs, and customer records used during development are **Stripe test-mode artefacts**. No live keys, no live cards, no real charges.

---

2. Phased Scope Isolation

2.1 Phase 1 — Core MVP (strictly essential for a working launch)

Phase 1 must produce a product that a founder can use **instead of** a spreadsheet for runway tracking.

In scope

#	Capability	Definition of Done
1	**Auth & onboarding**	Clerk sign-up/sign-in. First-run onboarding collects business name, starting cash, currency (single), and first month.
2	**Business workspace**	One or more businesses per account (tier-gated). Business switcher in the sidebar.

#	Capability	Definition of Done
3	**Category management**	User-editable revenue and expense categories with a `is_recurring` flag. Seeded with fictional defaults.
4	**Revenue & expense entry**	Line items per month: label, category, amount, recurrence (`one_off`, `monthly`, `custom`). Inline edit, keyboard-first.
5	**Monthly summaries**	Auto-derived opening cash, revenue, expenses, net cash flow, closing cash per month.
6	**Runway calculation**	Runway months + projected zero-cash month, with the formula displayed in plain language and a "show the maths" expansion.
7	**Cash-flow table**	6–36 month grid (tier-gated) with sticky header/first column and CSV export.
8	**Scenario planning**	Baseline scenario auto-created. Clone-to-scenario, edit assumptions, side-by-side comparison (up to tier limit).
9	**Runway warning banner**	Threshold-based banner (Critical < 3 months, Warning < 6 months) with contextual copy.
10	**Decision log**	Timestamped record of a decision: title, rationale, linked scenario, estimated monthly impact, status.
11	**Milestone timeline**	Dated business milestones (e.g. "Reach $5k MRR") displayed on a horizontal timeline.
12	**Notes**	Free-text notes attachable to a business, month, or decision.
13	**Billing**	Stripe Checkout (test mode) + webhook-driven subscription state. Tier gating enforced.
14	**Accessibility & privacy baseline**	WCAG 2.2 AA, consent capture, zero hardcoded secrets.

Out of scope for Phase 1 (must not be partially built)

Banking integrations, payroll, tax estimates, multi-currency, AI summaries, adviser client portal, invoicing, document uploads, mobile-native apps, real customer financial data.

MVP hard constraints

- **Fictional demo data only.** No real names, email addresses, phone numbers, physical addresses, or real financial figures. Seed data uses `@example.com`, `+852-0000-0000`, and names such as *Priya Demo Co.*
- **No banking integrations, no payroll, no advanced accounting** in Phase 1.
- **Calculations must be visible, simple, transparent, and editable.** No hidden constants. Default assumptions are surfaced in the UI and overridable.
- Data entry must be fast enough that a founder does not return to a spreadsheet — inline editing, keyboard navigation, and bulk month fill are launch-blocking.

2.2 Phase 2 — Post-Launch Expansion (deferred, do not build

now)

Feature	Why deferred	Notes for future phase
Banking / open-banking integrations	Requires regulated aggregator, real credentials, reconciliation logic.	Read-only transaction import feeding `revenue_items` / `expense_items`.
Payroll systems integration	High compliance surface, country-specific.	Out of scope until multi-region support exists.
Advanced accounting (double-entry, ledgers, P&L/BS/CF statements)	Contradicts "simple and transparent" positioning for MVP.	Optional "accountant export" bridge instead.
Tax estimates	Jurisdiction-dependent, liability risk.	Requires per-region rule engine + disclaimers.
Multi-currency support	Requires FX rate source, historical rates, conversion policy.	Store `currency_code` per business; add `fx_rates` table.
AI-generated financial summaries (Gemini API)	Not required for MVP; adds cost and trust risk before core accuracy is proven.	**Gemini API: NOT APPLICABLE to Phase 1.**
Adviser client portal	Requires invitation, granular permissions, audit trail.	Uses `team_members` table already defined in Phase 1.
Advanced forecasting models (seasonality, cohort, Monte Carlo)	Requires sufficient real data volume to be meaningful.	Layer on top of `scenario_assumptions`.
ImageKit / media pipeline	**NOT APPLICABLE to Phase 1** — BootPulse has no image, avatar-upload, or media-transformation requirement.	Re-evaluate only if receipts/invoice attachments are added.

---

3. Architecture & UI Layout

3.1 Page Routes (Next.js App Router)

```
/                                  → Marketing landing (hero, value props,
pricing table)
/pricing                           → Standalone pricing table (shared
component)
/legal/privacy                     → Privacy notice (GDPR / HK PDPO, PICS
disclaimer)
/legal/terms                       → Terms of service
/sign-in/[[...rest]]               → Clerk sign-in
/sign-up/[[...rest]]               → Clerk sign-up
```

```
/onboarding                              → 4-step first-run wizard (protected)

/app                                     → Redirect → most recent business
dashboard
/app/businesses                          → Business list + tier-gated "New
business"
/app/businesses/[businessId]             → Workspace dashboard (runway hero,
warnings, mini chart)
/app/businesses/[businessId]/cash-flow       → Cash-flow table + inline
editing
/app/businesses/[businessId]/scenarios       → Scenario comparison table
/app/businesses/[businessId]/scenarios/[scenarioId] → Scenario detail &
assumptions
/app/businesses/[businessId]/decisions       → Decision log
/app/businesses/[businessId]/milestones      → Milestone timeline
/app/businesses/[businessId]/notes           → Notes
/app/businesses/[businessId]/settings        → Categories, business
profile, danger zone
/app/billing                             → Current plan, usage vs limits, Stripe
portal link
/app/settings                            → Account, consent record, data
export/delete

/api/checkout                            → POST: create Stripe Checkout Session
/api/webhooks/stripe                     → POST: Stripe webhook listener
/api/businesses/[id]/export              → GET: CSV export of monthly summaries
```

Route protection: `/app/**` and `/onboarding` are guarded by Clerk middleware. Every server action and route handler re-validates ownership against the `businesses` row — UI gating alone is never trusted.

3.2 User Journey (Phase 1 happy path)

1. **Land** on `/` → hero states the runway promise → pricing table → CTA "Start free trial".
2. **Sign up** via Clerk → redirected to `/onboarding`.
3. **Onboarding wizard**
   - Step 1: Business name + currency + business start month.
   - Step 2: Starting cash available today.
   - Step 3: Add 2–5 revenue lines and 2–5 expense lines (pre-filled with editable fictional examples).
   - Step 4: Confirm assumptions (trailing burn window, runway warning thresholds) → "Create my dashboard".
4. **Land on workspace dashboard** → runway hero figure, zero-cash month, warning banner if applicable.
5. **Review cash-flow table** → adjust a figure inline → runway recalculates instantly.
6. **Clone to scenario** → "Hire a freelancer at $2,000/month from month 3" → save.
7. **Compare** baseline vs scenario side-by-side → runway delta highlighted.

8. **Record decision** → decision log entry linked to the scenario.
9. **Hit a tier limit** → upgrade prompt → Stripe Checkout (test mode) → webhook updates tier → limit lifted.

3.3 Key UI Components

Component	Location	Responsibility	Key props / states
`HeroSection`	`/`, `/pricing`	Headline value prop, primary CTA, product screenshot placeholder.	`variant: "landing" | "pricing"`
`WorkspaceDashboard`	`/app/businesses/[id]`	Composes runway hero, warnings, chart, and recent activity.	`businessId`, `summary`
`RunwayHero`	Dashboard	Large runway-months figure + zero-cash month + "Show the maths" disclosure.	`runwayMonths`, `zeroCashMonth`, `formulaTrace`
`RunwayWarningBanner`	Dashboard, all workspace pages	Threshold-driven banner.	`severity: "critical" | "warning" | "healthy"`, `dismissible` (session-only)
`CashFlowTable`	`/cash-flow`	Month columns, sticky first column, editable cells, totals row.	`months[]`, `editable`, `onCellCommit`, `tierHorizon`
`ScenarioComparisonTable`	`/scenarios`	Baseline vs up to N scenarios, per-metric rows, delta column.	`scenarios[]`, `baselineId`, `metrics[]`, `maxScenarios`
`ScenarioAssumptionEditor`	`/scenarios/[id]`	Add/edit assumption rows (label, type, amount, start month, end month).	`assumptions[]`, `onChange`
`DecisionLog`	`/decisions`	Reverse-chronological list with status chips and linked scenario.	`decisions[]`, `onCreate`, `onStatusChange`
`MilestoneTimeline`	`/milestones`	Horizontal dated timeline with past/future states.	`milestones[]`, `today`
`PricingTable`	`/pricing`, `/billing`	Three tiers, feature matrix, current-plan marker.	`tiers[]`, `currentTier`, `onSelect`

Component	Location	Responsibility	Key props / states
`FormulaDisclosure`	Inline everywhere a number is computed	Expands to show inputs + arithmetic in plain language.	`label`, `expression`, `inputs[]`
`TierLimitCallout`	Any gated surface	Explains the limit and links to `/billing`.	`limitType`, `currentTier`, `requiredTier`
`EmptyState`	All list views	Explains the next action with a single primary button.	`title`, `body`, `cta`

3.4 Layout Rules (Desktop-First)

- **Minimum supported viewport:** 1280 × 720. Optimised for 1440 × 900 and above.
- **Below 1024px:** show a read-only summary view with headline runway figures and a notice that detailed planning requires a larger screen. No table editing on narrow viewports.
- **Below 768px:** informational page only — no workspace editing.
- **Grid:** 12-column, max content width 1440px, sidebar 240px fixed, sticky table headers and sticky first column.
- **No horizontal page scroll** at ≥1280px; wide tables scroll within their own container.

---

4. Technical Stack Constraints

Layer	Technology	Constraint / usage in BootPulse
Framework	**Next.js (App Router)**	Server Components by default. Server Actions for all mutations. Route handlers only for checkout, webhooks, and export.
Language	**TypeScript**	`strict: true`. No `any` in application code. All money values typed as branded `MinorUnits = number & { __brand: 'MinorUnits' }`.
Styling	**Tailwind CSS**	Design tokens via `tailwind.config.ts`. No ad-hoc hex values in components — use token classes only.
ORM	**Drizzle ORM**	Schema-first in `src/db/schema.ts`. Migrations via `drizzle-kit`. All queries parameterised; no raw string interpolation.
Database	**Neon PostgreSQL**	Serverless Postgres. Connection pooling via Neon's pooled connection string. RLS enabled on every tenant table.
Auth	**Clerk**	Middleware-protected routes. Clerk `userId` is the sole identity source; mapped to `users.clerk_user_id`.

Layer	Technology	Constraint / usage in BootPulse
Payments	**Stripe Checkout**	**Test mode only.** Checkout Sessions created server-side. Entitlements driven exclusively by webhooks.
ImageKit	**NOT APPLICABLE — Phase 1**	BootPulse MVP has no image upload, avatar, or media transformation requirement. Do not integrate.
Gemini API	**NOT APPLICABLE — Phase 1**	AI-generated financial summaries are a Phase 2 feature. Do not integrate in MVP.

4.1 Money & Precision Rules

- All monetary values stored as **integer minor units** (`integer` / `bigint` columns) — never floats.
- Rounding happens once, at display time, using a shared `formatMoney(minorUnits, currency)` helper.
- All date/month values stored as `date` (first day of month) in UTC. No local-time arithmetic for financial periods.

4.2 Core Calculation Contract (must be visible in the UI)

```
net_cash_flow(month)   = revenue_total(month) − expense_total(month)
closing_cash(month)    = opening_cash(month) + net_cash_flow(month)
opening_cash(month+1)  = closing_cash(month)
opening_cash(first)    = business.starting_cash
avg_monthly_burn       = mean( max(0, −net_cash_flow(m)) ) over trailing
window (default 3 months)
runway_months          = avg_monthly_burn > 0 ? current_cash /
avg_monthly_burn : "Cash-flow positive"
zero_cash_month        = first month where closing_cash < 0
```

Every one of these is rendered through `FormulaDisclosure` so the user can see the inputs and the arithmetic. The trailing-burn window and warning thresholds are **user-editable** in Settings.

---

5. Database & Authentication Schema

5.1 Conventions

- **Primary keys:** `uuid` with `defaultRandom()` (`gen_random_uuid()`).
- **Foreign keys:** always `ON DELETE CASCADE` for tenant-owned data.
- **Timestamps:** `timestamp with time zone`, `defaultNow()`, `notNull()`.
- **Soft delete:** not used in Phase 1 — cascading hard deletes plus an explicit confirmation dialog.
- **Enums:** Postgres enums via Drizzle `pgEnum`.
- **Every tenant table carries `business_id`** so RLS can be expressed uniformly.

5.2 Enums

```
export const planTierEnum      = pgEnum('plan_tier', ['starter', 'growth',
'scale']);
export const subStatusEnum     = pgEnum('subscription_status',
  ['trialing', 'active', 'past_due', 'canceled', 'incomplete']);
export const memberRoleEnum    = pgEnum('member_role', ['owner', 'editor',
'viewer']);
export const memberStatusEnum  = pgEnum('member_status', ['invited',
'active', 'revoked']);
export const itemTypeEnum      = pgEnum('item_type', ['revenue',
'expense']);
export const recurrenceEnum    = pgEnum('recurrence', ['one_off', 'monthly',
'custom']);
export const scenarioKindEnum  = pgEnum('scenario_kind', ['baseline',
'scenario']);
export const assumptionTypeEnum= pgEnum('assumption_type',
  ['add_expense', 'remove_expense', 'change_revenue', 'one_off_cost',
'one_off_gain']);
export const decisionStatusEnum= pgEnum('decision_status',
  ['considering', 'committed', 'rejected', 'revisited']);
export const milestoneStateEnum= pgEnum('milestone_state', ['planned',
'achieved', 'missed']);
export const warningLevelEnum  = pgEnum('warning_level', ['critical',
'warning', 'healthy']);
```

5.3 Tables (Drizzle — `src/db/schema.ts`)

```
// ---------- users ----------
export const users = pgTable('users', {
  id:              uuid('id').primaryKey().defaultRandom(),
  clerkUserId:     text('clerk_user_id').notNull().unique(),
  displayName:     text('display_name').notNull(),          // fictional in
demo data
  contactEmail:    text('contact_email').notNull(),         // @example.com
in demo data
  planTier:        planTierEnum('plan_tier').notNull().default('starter'),
  trialEndsAt:     timestamp('trial_ends_at', { withTimezone: true }),
  consentPrivacyAt:timestamp('consent_privacy_at', { withTimezone: true }),
  consentTermsAt:  timestamp('consent_terms_at',   { withTimezone: true }),
  marketingOptIn:  boolean('marketing_opt_in').notNull().default(false),
  createdAt:       timestamp('created_at', { withTimezone: true
}).notNull().defaultNow(),
  updatedAt:       timestamp('updated_at', { withTimezone: true
}).notNull().defaultNow(),
});

// ---------- businesses ----------
export const businesses = pgTable('businesses', {
```

```
  id:               uuid('id').primaryKey().defaultRandom(),
  ownerUserId:      uuid('owner_user_id').notNull()
                      .references(() => users.id, { onDelete: 'cascade' }),
  name:             text('name').notNull(),
  currencyCode:     text('currency_code').notNull().default('USD'),   //
single currency, Phase 1
  startingCashMinor:integer('starting_cash_minor').notNull().default(0),
  startMonth:       date('start_month').notNull(),
  forecastMonths:   integer('forecast_months').notNull().default(6),
  burnWindowMonths: integer('burn_window_months').notNull().default(3),
  criticalThresholdMonths:
integer('critical_threshold_months').notNull().default(3),
  warningThresholdMonths:
integer('warning_threshold_months').notNull().default(6),
  stripeCustomerId: text('stripe_customer_id'),                     //
test mode only
  createdAt:        timestamp('created_at', { withTimezone: true
}).notNull().defaultNow(),
  updatedAt:        timestamp('updated_at', { withTimezone: true
}).notNull().defaultNow(),
}, (t) => ({ ownerIdx: index('businesses_owner_idx').on(t.ownerUserId) }));

// ---------- team_members ----------
export const teamMembers = pgTable('team_members', {
  id:           uuid('id').primaryKey().defaultRandom(),
  businessId:   uuid('business_id').notNull()
                  .references(() => businesses.id, { onDelete: 'cascade' }),
  userId:       uuid('user_id').references(() => users.id, { onDelete:
'cascade' }),
  invitedEmail: text('invited_email'),                   // @example.com in
demo data
  role:         memberRoleEnum('role').notNull().default('viewer'),
  status:       memberStatusEnum('status').notNull().default('invited'),
  invitedAt:    timestamp('invited_at', { withTimezone: true
}).notNull().defaultNow(),
  acceptedAt:   timestamp('accepted_at', { withTimezone: true }),
  createdAt:    timestamp('created_at', { withTimezone: true
}).notNull().defaultNow(),
}, (t) => ({ bizIdx: index('team_members_business_idx').on(t.businessId)
}));

// ---------- categories ----------
export const categories = pgTable('categories', {
  id:         uuid('id').primaryKey().defaultRandom(),
  businessId: uuid('business_id').notNull()
                .references(() => businesses.id, { onDelete: 'cascade' }),
  name:       text('name').notNull(),
  itemType:   itemTypeEnum('item_type').notNull(),
  colorToken: text('color_token').notNull().default('neutral'),
  isRecurring:boolean('is_recurring').notNull().default(false),
  sortOrder:  integer('sort_order').notNull().default(0),
  createdAt:  timestamp('created_at', { withTimezone: true
```

```
}).notNull().defaultNow(),
}, (t) => ({ bizIdx: index('categories_business_idx').on(t.businessId) }));

// ---------- revenue_items ----------
export const revenueItems = pgTable('revenue_items', {
  id:           uuid('id').primaryKey().defaultRandom(),
  businessId:   uuid('business_id').notNull()
                  .references(() => businesses.id, { onDelete: 'cascade' }),
  categoryId:   uuid('category_id').references(() => categories.id, {
onDelete: 'set null' }),
  label:        text('label').notNull(),
  amountMinor:  integer('amount_minor').notNull(),
  recurrence:   recurrenceEnum('recurrence').notNull().default('monthly'),
  startMonth:   date('start_month').notNull(),
  endMonth:     date('end_month'),
  growthRateBps:integer('growth_rate_bps').notNull().default(0), // basis
points, editable assumption
  notes:        text('notes'),
  createdAt:    timestamp('created_at', { withTimezone: true
}).notNull().defaultNow(),
  updatedAt:    timestamp('updated_at', { withTimezone: true
}).notNull().defaultNow(),
}, (t) => ({ bizIdx: index('revenue_items_business_idx').on(t.businessId)
}));

// ---------- expense_items ----------
export const expenseItems = pgTable('expense_items', {
  id:           uuid('id').primaryKey().defaultRandom(),
  businessId:   uuid('business_id').notNull()
                  .references(() => businesses.id, { onDelete: 'cascade' }),
  categoryId:   uuid('category_id').references(() => categories.id, {
onDelete: 'set null' }),
  label:        text('label').notNull(),
  amountMinor:  integer('amount_minor').notNull(),
  recurrence:   recurrenceEnum('recurrence').notNull().default('monthly'),
  startMonth:   date('start_month').notNull(),
  endMonth:     date('end_month'),
  growthRateBps:integer('growth_rate_bps').notNull().default(0),
  notes:        text('notes'),
  createdAt:    timestamp('created_at', { withTimezone: true
}).notNull().defaultNow(),
  updatedAt:    timestamp('updated_at', { withTimezone: true
}).notNull().defaultNow(),
}, (t) => ({ bizIdx: index('expense_items_business_idx').on(t.businessId)
}));

// ---------- monthly_summaries ----------
export const monthlySummaries = pgTable('monthly_summaries', {
  id:             uuid('id').primaryKey().defaultRandom(),
  businessId:     uuid('business_id').notNull()
                    .references(() => businesses.id, { onDelete: 'cascade'
}),
```

```
  scenarioId:     uuid('scenario_id')
                    .references(() => scenarios.id, { onDelete: 'cascade'
}),
  month:          date('month').notNull(),
  openingCashMinor: integer('opening_cash_minor').notNull().default(0),
  revenueMinor:   integer('revenue_minor').notNull().default(0),
  expenseMinor:   integer('expense_minor').notNull().default(0),
  netCashFlowMinor: integer('net_cash_flow_minor').notNull().default(0),
  closingCashMinor: integer('closing_cash_minor').notNull().default(0),
  runwayMonths:   numeric('runway_months', { precision: 6, scale: 2 }),
  warningLevel:   warningLevelEnum('warning_level'),
  computedAt:     timestamp('computed_at', { withTimezone: true
}).notNull().defaultNow(),
}, (t) => ({
  bizIdx:      index('monthly_summaries_business_idx').on(t.businessId),
  uniqueMonth: uniqueIndex('monthly_summaries_biz_scenario_month_uq')
                 .on(t.businessId, t.scenarioId, t.month),
}));

// ---------- scenarios ----------
export const scenarios = pgTable('scenarios', {
  id:           uuid('id').primaryKey().defaultRandom(),
  businessId:   uuid('business_id').notNull()
                  .references(() => businesses.id, { onDelete: 'cascade' }),
  name:         text('name').notNull(),
  kind:         scenarioKindEnum('kind').notNull().default('scenario'),
  description:  text('description'),
  horizonMonths:integer('horizon_months').notNull().default(6),
  createdByUserId: uuid('created_by_user_id')
                  .references(() => users.id, { onDelete: 'set null' }),
  isArchived:   boolean('is_archived').notNull().default(false),
  createdAt:    timestamp('created_at', { withTimezone: true
}).notNull().defaultNow(),
  updatedAt:    timestamp('updated_at', { withTimezone: true
}).notNull().defaultNow(),
}, (t) => ({ bizIdx: index('scenarios_business_idx').on(t.businessId) }));

// ---------- scenario_assumptions ----------
export const scenarioAssumptions = pgTable('scenario_assumptions', {
  id:           uuid('id').primaryKey().defaultRandom(),
  scenarioId:   uuid('scenario_id').notNull()
                  .references(() => scenarios.id, { onDelete: 'cascade' }),
  assumptionType: assumptionTypeEnum('assumption_type').notNull(),
  label:        text('label').notNull(),
  amountMinor:  integer('amount_minor').notNull(),
  startMonth:   date('start_month').notNull(),
  endMonth:     date('end_month'),
  isEnabled:    boolean('is_enabled').notNull().default(true),
  sortOrder:    integer('sort_order').notNull().default(0),
  createdAt:    timestamp('created_at', { withTimezone: true
}).notNull().defaultNow(),
}, (t) => ({ scenarioIdx:
```

```
index('scenario_assumptions_scenario_idx').on(t.scenarioId) }));

// ---------- milestones ----------
export const milestones = pgTable('milestones', {
  id:          uuid('id').primaryKey().defaultRandom(),
  businessId:  uuid('business_id').notNull()
                 .references(() => businesses.id, { onDelete: 'cascade' }),
  title:       text('title').notNull(),
  description: text('description'),
  targetMonth: date('target_month').notNull(),
  state:       milestoneStateEnum('state').notNull().default('planned'),
  createdAt:   timestamp('created_at', { withTimezone: true
}).notNull().defaultNow(),
  updatedAt:   timestamp('updated_at', { withTimezone: true
}).notNull().defaultNow(),
}, (t) => ({ bizIdx: index('milestones_business_idx').on(t.businessId) }));

// ---------- decisions ----------
export const decisions = pgTable('decisions', {
  id:              uuid('id').primaryKey().defaultRandom(),
  businessId:      uuid('business_id').notNull()
                     .references(() => businesses.id, { onDelete: 'cascade'
}),
  scenarioId:      uuid('scenario_id')
                     .references(() => scenarios.id, { onDelete: 'set null'
}),
  title:           text('title').notNull(),
  rationale:       text('rationale'),
  estimatedMonthlyImpactMinor:
integer('estimated_monthly_impact_minor').notNull().default(0),
  status:
decisionStatusEnum('status').notNull().default('considering'),
  decidedAt:       timestamp('decided_at', { withTimezone: true }),
  createdAt:       timestamp('created_at', { withTimezone: true
}).notNull().defaultNow(),
  updatedAt:       timestamp('updated_at', { withTimezone: true
}).notNull().defaultNow(),
}, (t) => ({ bizIdx: index('decisions_business_idx').on(t.businessId) }));

// ---------- notes ----------
export const notes = pgTable('notes', {
  id:            uuid('id').primaryKey().defaultRandom(),
  businessId:    uuid('business_id').notNull()
                   .references(() => businesses.id, { onDelete: 'cascade'
}),
  decisionId:    uuid('decision_id')
                   .references(() => decisions.id, { onDelete: 'cascade' }),
  month:         date('month'),
  body:          text('body').notNull(),
  createdByUserId: uuid('created_by_user_id')
                   .references(() => users.id, { onDelete: 'set null' }),
  createdAt:     timestamp('created_at', { withTimezone: true
```

```
}).notNull().defaultNow(),
}, (t) => ({ bizIdx: index('notes_business_idx').on(t.businessId) }));
```

5.4 Row Level Security

RLS is enabled on **every** tenant table. Identity is resolved from the Clerk JWT `sub` claim, which Neon exposes via `request.jwt.claims`.

```
-- Identity helper: maps the Clerk JWT subject to an internal user id.
create or replace function app_current_user_id()
returns uuid
language sql
stable
as $$
  select id
  from users
  where clerk_user_id = (current_setting('request.jwt.claims', true)::json
->> 'sub')
$$;

-- Membership helper: true when the caller owns or belongs to the business.
create or replace function app_has_business_access(target_business_id uuid)
returns boolean
language sql
stable
as $$
  select exists (
    select 1 from businesses b
    where b.id = target_business_id
      and b.owner_user_id = app_current_user_id()
  ) or exists (
    select 1 from team_members tm
    where tm.business_id = target_business_id
      and tm.user_id = app_current_user_id()
      and tm.status = 'active'
  )
$$;

alter table users               enable row level security;
alter table businesses          enable row level security;
alter table team_members        enable row level security;
alter table categories          enable row level security;
alter table revenue_items       enable row level security;
alter table expense_items       enable row level security;
alter table monthly_summaries   enable row level security;
alter table scenarios           enable row level security;
alter table scenario_assumptions enable row level security;
alter table milestones          enable row level security;
alter table decisions           enable row level security;
alter table notes               enable row level security;
```

```
-- users: read/update self only
create policy users_self_select on users
  for select using (clerk_user_id = (current_setting('request.jwt.claims',
true)::json ->> 'sub'));
create policy users_self_update on users
  for update using (clerk_user_id = (current_setting('request.jwt.claims',
true)::json ->> 'sub'));

-- businesses: owner full control; members read-only
create policy businesses_owner_all on businesses
  for all using (owner_user_id = app_current_user_id())
  with check (owner_user_id = app_current_user_id());
create policy businesses_member_select on businesses
  for select using (app_has_business_access(id));

-- Generic tenant tables (repeat per table, substituting the table name)
create policy team_members_access on team_members
  for all using (app_has_business_access(business_id))
  with check (app_has_business_access(business_id));

create policy categories_access on categories
  for all using (app_has_business_access(business_id))
  with check (app_has_business_access(business_id));

create policy revenue_items_access on revenue_items
  for all using (app_has_business_access(business_id))
  with check (app_has_business_access(business_id));

create policy expense_items_access on expense_items
  for all using (app_has_business_access(business_id))
  with check (app_has_business_access(business_id));

create policy monthly_summaries_access on monthly_summaries
  for all using (app_has_business_access(business_id))
  with check (app_has_business_access(business_id));

create policy scenarios_access on scenarios
  for all using (app_has_business_access(business_id))
  with check (app_has_business_access(business_id));

create policy scenario_assumptions_access on scenario_assumptions
  for all using (exists (
    select 1 from scenarios s
    where s.id = scenario_assumptions.scenario_id
      and app_has_business_access(s.business_id)
  ))
  with check (exists (
    select 1 from scenarios s
    where s.id = scenario_assumptions.scenario_id
      and app_has_business_access(s.business_id)
  ));
```

```
create policy milestones_access on milestones
  for all using (app_has_business_access(business_id))
  with check (app_has_business_access(business_id));

create policy decisions_access on decisions
  for all using (app_has_business_access(business_id))
  with check (app_has_business_access(business_id));

create policy notes_access on notes
  for all using (app_has_business_access(business_id))
  with check (app_has_business_access(business_id));
```

RLS notes

- The application connects with a **non-superuser, non-BYPASSRLS** role. Migrations run under a separate owner role.
- Every request sets the Clerk JWT on the connection (`SET LOCAL request.jwt.claims = ...`) inside a transaction before querying.
- Tier gating is a **second, independent check** in the server action layer — RLS governs *whose* data, application code governs *how much* data.

---

6. Third-Party API & Payment Boundaries

6.1 Required Environment Keys — `.env.local`

```
# ---- Database (Neon PostgreSQL) ----
DATABASE_URL="postgresql://<user>:<password>@<pooled-
host>/<db>?sslmode=require"
DATABASE_URL_UNPOOLED="postgresql://<user>:<password>@<direct-
host>/<db>?sslmode=require"

# ---- Clerk Auth ----
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY="pk_test_..."
CLERK_SECRET_KEY="sk_test_..."
CLERK_WEBHOOK_SIGNING_SECRET="whsec_..."
NEXT_PUBLIC_CLERK_SIGN_IN_URL="/sign-in"
NEXT_PUBLIC_CLERK_SIGN_UP_URL="/sign-up"
NEXT_PUBLIC_CLERK_AFTER_SIGN_IN_URL="/app"
NEXT_PUBLIC_CLERK_AFTER_SIGN_UP_URL="/onboarding"

# ---- Stripe (TEST MODE ONLY) ----
STRIPE_SECRET_KEY="sk_test_..."
STRIPE_WEBHOOK_SECRET="whsec_..."
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY="pk_test_..."
STRIPE_PRICE_STARTER_MONTHLY="price_..."   # test-mode price id
STRIPE_PRICE_GROWTH_MONTHLY="price_..."
STRIPE_PRICE_SCALE_MONTHLY="price_..."

# ---- App ----
```

```
NEXT_PUBLIC_APP_URL="http://localhost:3000"

# ---- Phase 2 only — NOT REQUIRED FOR MVP ----
# GEMINI_API_KEY=""        # NOT APPLICABLE in Phase 1
# IMAGEKIT_PUBLIC_KEY=""   # NOT APPLICABLE in Phase 1
# IMAGEKIT_PRIVATE_KEY=""  # NOT APPLICABLE in Phase 1
# IMAGEKIT_URL_ENDPOINT="" # NOT APPLICABLE in Phase 1
```

Rules

- `.env.local` is git-ignored. A committed `.env.example` lists key **names only**, never values.
- Secrets are read **only** in server-side code (`process.env`, no NEXT_PUBLIC_ prefix) except the two publishable keys, which are safe by design.
- **Never hardcode any secret, key, price ID, or webhook secret** in source, fixtures, or tests. CI fails on secret-scanning violations.

6.2 Stripe Checkout Session Triggers

Trigger	Route	Behaviour
User clicks **Upgrade** on `/billing` or a `TierLimitCallout`	`POST /api/checkout`	Server verifies the Clerk session, resolves the target `price_id` from a **server-side tier→price map**, reuses/creates the Stripe test customer, and creates a Checkout Session in `mode: 'subscription'`.
Checkout completes	Stripe redirect	Returns to `/billing?checkout=success`. **Entitlement is NOT granted here.** UI shows a "finalising your plan" state until the webhook lands.
User clicks **Manage billing**	`POST /api/billing/portal`	Creates a Stripe Billing Portal session for the test customer.
Checkout abandoned	Stripe redirect	Returns to `/billing?checkout=cancelled`. No state change.

Checkout Session creation contract

```
mode: 'subscription'
customer: <stripe_customer_id | newly created test customer>
line_items: [{ price: <resolved test price id>, quantity: 1 }]
client_reference_id: <internal user uuid>
metadata: { user_id, business_id, target_tier }
success_url:
`${APP_URL}/billing?checkout=success&session_id={CHECKOUT_SESSION_ID}`
cancel_url:  `${APP_URL}/billing?checkout=cancelled`
allow_promotion_codes: true
```

The tier→price mapping lives in `src/lib/stripe/prices.ts` on the server. The client sends a **tier name**, never a price ID.

6.3 Webhook Listeners

Endpoint: `POST /api/webhooks/stripe`
Verification: `stripe.webhooks.constructEvent(rawBody, signature, STRIPE_WEBHOOK_SECRET)` against the **raw** request body. Reject with `400` on failure. Never parse the body before verification.

Event	Action
`checkout.session.completed`	Resolve `client_reference_id` → user. Persist `stripe_customer_id` and `stripe_subscription_id`. Set `plan_tier` from `metadata.target_tier`. Set `subscription_status = 'active'` (or `'trialing'`).
`customer.subscription.created`	Upsert subscription record; sync `plan_tier` and period end.
`customer.subscription.updated`	Re-sync `plan_tier`, status, and `current_period_end`. Downgrades set data above the new limit to read-only — **never delete**.
`customer.subscription.deleted`	Set `subscription_status = 'canceled'`, revert to `starter` limits, keep all data.
`invoice.payment_failed`	Set `subscription_status = 'past_due'`, surface an in-app banner, retain entitlements for a 7-day grace period.
`invoice.payment_succeeded`	Clear `past_due`, extend `current_period_end`.

Webhook rules

- **Idempotent:** every handler records the Stripe `event.id` in a `processed_webhook_events` table and exits early on replay.
- Respond `200` quickly; long work is queued, not awaited inline.
- Only `stripe` test-mode events are accepted. Live-mode keys are never present in any environment for this assignment.
- **Entitlements are webhook-authoritative.** A `?checkout=success` query param never unlocks a tier.

---

7. Compliance & Security

7.1 Accessibility — WCAG 2.2 Level AA

Requirement	Implementation standard
Text contrast	Body and UI text contrast ratio **≥ 4.5:1**; large text (≥ 24px, or ≥ 19px bold) **≥ 3:1**. Enforced as Tailwind token pairs; a CI contrast check validates the token palette.

Requirement	Implementation standard
Focus visibility	Every interactive element has a visible focus indicator: **`outline: 2px solid`** with a 2px offset, using the `--focus-ring` token. `outline: none` is prohibited without an equivalent replacement. Focus is never obscured (2.4.11).
Keyboard operability	All editing — table cells, scenario assumptions, decision log — is fully keyboard accessible. `Tab` moves between cells, `Enter` commits, `Escape` reverts. No keyboard traps.
Target size	Interactive targets ≥ 24 × 24 CSS px (2.5.8); primary actions ≥ 44 × 44 px.
Consistent help	Help affordances appear in the same relative location on every workspace page (2.4.13).
Redundant entry	Previously entered values are auto-populated or selectable within the same session (3.3.7).
Accessible authentication	No cognitive-function test for sign-in; paste and password managers are supported (3.3.8). Clerk handles this.
Semantics	Landmarks (`header`, `nav`, `main`, `aside`), one `h1` per page, logical heading order, real `<table>` markup with `<th scope>` for the cash-flow table.
Colour independence	Runway severity is conveyed by **icon + text label**, not colour alone.
Charts & data	Every chart has a text/table equivalent. Charts are `aria-hidden` with an adjacent accessible data table.
Motion	All animation respects `prefers-reduced-motion`.
Forms	Programmatic labels, `aria-describedby` for error text, errors announced via `role="alert"`.

Verification: axe-core in CI on every route, plus a manual keyboard-and-screen-reader pass (VoiceOver + NVDA) before launch.

7.2 Privacy — GDPR & Hong Kong PDPO

Obligation	Implementation
Lawful basis & notice	Privacy notice at `/legal/privacy` written in plain language, covering GDPR Art. 13/14 and PDPO Data Protection Principle 1.
PICS disclaimer	On `/legal/privacy` and at the point of collection, a Personal Information Collection Statement states: purpose of collection, classes of transferees (Clerk, Neon, Stripe), retention period, and the right to access and correct. Explicit PDPO-compliant wording, not generic GDPR boilerplate.
Consent timestamps	`users.consent_privacy_at` and `users.consent_terms_at` are written server-side at sign-up. Marketing consent is **separate, unticked by default**, recorded as `users.marketing_opt_in`. Consent events are append-only and never back-dated.

Obligation	Implementation
Data minimisation	Only collect what the product needs: display name, contact email, financial entries. No bank credentials, no ID documents, no payment card data (Stripe Checkout hosts the card form; card data never touches BootPulse servers).
Right of access / portability	`/app/settings` offers one-click **JSON + CSV export** of all user data.
Right to erasure	`/app/settings` offers account deletion with a typed confirmation. Deletion cascades through every tenant table via `ON DELETE CASCADE`. Stripe customer records are deleted via the Stripe test API; the request is logged.
Cross-border transfer	Document Neon and Clerk regions; rely on Standard Contractual Clauses / appropriate safeguards, disclosed in the privacy notice.
Retention	Active data retained while the account exists. On deletion, data is removed within 30 days; backups expire within 90 days.
Breach notification	Documented internal procedure; GDPR 72-hour and PDPO prompt-notification pathways defined.
Demo data rule	Phase 1 ships **fictional demo data only** — no real names, emails, phone numbers, addresses, or financial figures. Demo emails use `@example.com`; demo phone numbers use reserved ranges.

7.3 Secret & Application Security

- **Zero hardcoded secrets.** No keys, tokens, price IDs, or connection strings in source, tests, fixtures, seed scripts, or comments.
- **Secret scanning** (e.g. gitleaks) runs in CI and blocks merges on detection.
- **Server-only secrets** never carry the `NEXT_PUBLIC_` prefix and are never imported into Client Components.
- **All mutations go through Server Actions or Route Handlers** that re-authenticate the Clerk session and re-authorise business ownership — client-supplied IDs are never trusted.
- **Input validation** with Zod at every trust boundary (Server Actions, Route Handlers, webhook payloads after signature verification).
- **SQL injection prevention** via Drizzle's parameterised query builder; raw SQL only for the reviewed RLS helper functions.
- **Security headers** (`Content-Security-Policy`, `Strict-Transport-Security`, `X-Content-Type-Options`, `Referrer-Policy`, `Permissions-Policy`) configured in `next.config.ts`.
- **Rate limiting** on `/api/checkout` and `/api/webhooks/stripe`.
- **Least privilege:** the runtime DB role is not a superuser and does not hold BYPASSRLS.
- **Audit trail:** tier changes, consent events, and account deletions are logged with timestamps and actor IDs.

8. Phase 1 Acceptance Criteria (Launch Gate)

1. A new user can sign up, complete onboarding, and see a calculated runway figure in **under 3 minutes**.
2. Every displayed financial figure exposes its formula via `FormulaDisclosure`.
3. Editing any cash-flow cell recalculates runway and the zero-cash month in **under 300ms**.
4. A user can clone the baseline into a scenario, add "hire a freelancer at $2,000/month from month 3", and see the runway delta side-by-side.
5. Tier limits block creation beyond the plan, and upgrading via **Stripe test mode** lifts the limit **only after** the webhook is processed.
6. `axe-core` reports **zero critical or serious** violations on all Phase 1 routes.
7. All text meets **≥ 4.5:1** contrast; all focusable elements render a **2px solid** focus ring.
8. Secret scanning passes with zero findings; no live Stripe keys exist in any environment.
9. Account deletion removes all tenant rows via cascade and completes within 30 days.
10. All shipped demo content is fictional and uses reserved example domains and number ranges.